

Entrega 1 CVI

Created by  Andrea

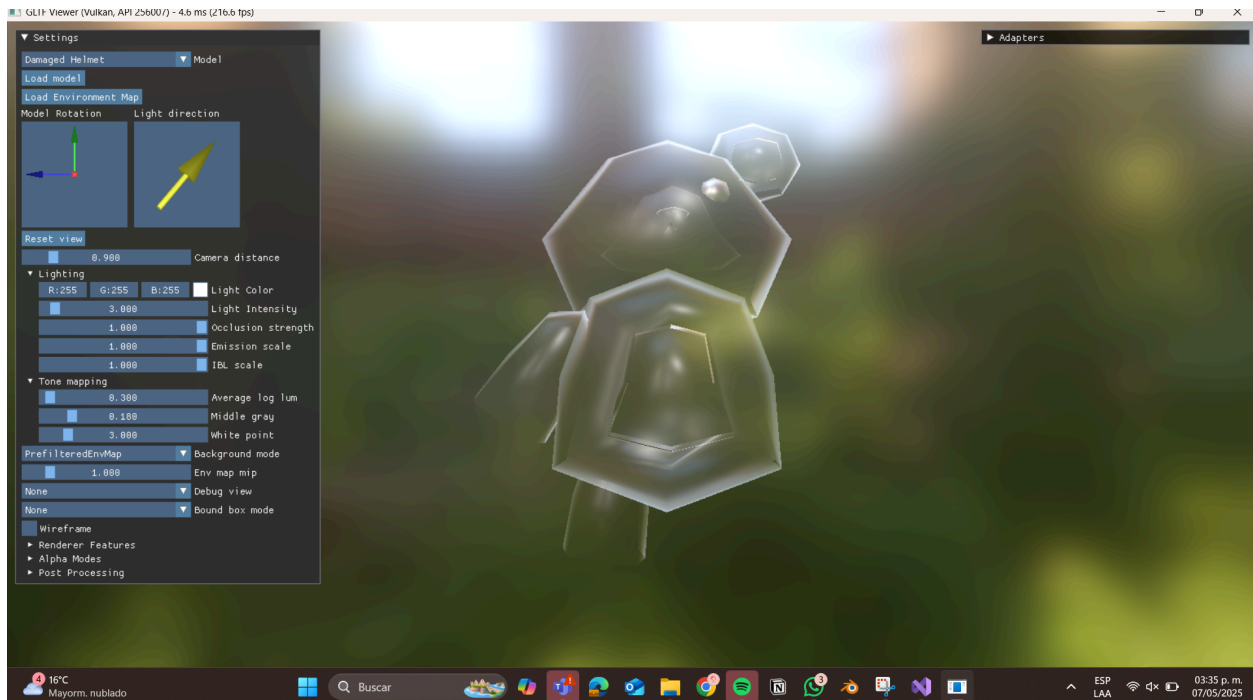
Idea General

La idea del proyecto consiste en realizar un oso de goma en realidad aumentada con el cual se pueda interactuar por medio de esta.

Avance Entrega 1

Para esta entrega se realizo el oso con textura de goma en Blender, de manera que se vea una textura de goma para poder importarlo al ejemplo de Diligent de GLTF, con base a este ejemplo se puede evidenciar diferentes obstaculos con la idea inicial debido a que al importarlo al ejemplo de GLTF ocurren 2 problemas: Inicialmente se ve el oso pero sin textura o se ve el oso de manera extraña dentro del ejemplo faltándole partes y teniendo una geometria distinta a la hecha de manera inicial

[attachment:47235337-1021-4377-8d0d-08f473a9abd9:Grabacin_de_pantalla_2025-05-07_152147.mp4](#)



Debido a estos problemas vistos en la imagen anterior se empezó a ver el código de este programa para poder ingresar la textura ya dentro del loader sin necesidad de utilizar el load model dentro de el programa.

y se establecio que las texturas que se van a usar para el desarrollo del proyecto van a ser las siguientes:

este ejemplo fue tomado del ejemplo de tutorial 21 y con base a este se va a utilizar una textura para subir al oso cuando este este definido de manera correcta para que de esta maenra este pueda tener una apariencia cristalina sin hacelro directamente con los shaders

```
#include "structures.fxh"
#include "RayUtils.fxh"

ConstantBuffer<CubeAttribs> g_CubeAttribsCB;

// Simulate light absorption inside glass.
float3 LightAbsorption(float3 color1, float depth)
{
    float factor1 = depth * 0.25;
    float factor2 = pow(depth * g_ConstantsCB.GlassAbsorption, 2.2) * 0.25;
    float factor = clamp(factor1 + factor2 + 0.05, 0.0, 1.0);
    float3 color2 = color1 * g_ConstantsCB.GlassMaterialColor.rgb;
    return lerp(color1, color2, factor);
}

float3 BlendWithReflection(float3 srcColor, float3 reflectionColor, float factor)
{
    return lerp(srcColor, reflectionColor * g_ConstantsCB.GlassReflectionColorMa:
}

// Optimized fresnel calculation.
float Fresnel(float eta, float cosThetal)
{
    cosThetal = clamp(cosThetal, -1.0, 1.0);
    if (cosThetal < 0.0)
    {
        eta = 1.0 / eta;
        cosThetal = -cosThetal;
    }
}
```

```

float sinThetaTSq = eta * eta * (1.0 - cosThetal * cosThetal);
if (sinThetaTSq > 1.0)
    return 1.0;

float cosThetaT = sqrt(1.0 - sinThetaTSq);

float Rs = (eta * cosThetal - cosThetaT) / (eta * cosThetal + cosThetaT);
float Rp = (eta * cosThetaT - cosThetal) / (eta * cosThetaT + cosThetal);

return 0.5 * (Rs * Rs + Rp * Rp);
}

[shader("closesthit")]
void main(inout PrimaryRayPayload payload, in BuiltInTriangleIntersectionAttribut
{
    // Calculate triangle barycentrics.
    float3 barycentrics = float3(1.0 - attr.barycentrics.x - attr.barycentrics.y, attr.barycentrics.z);

    // Get vertex indices for primitive.
    uint3 primitive = g_CubeAttribsCB.Primitives[PrimitiveIndex()].xyz;

    // Calculate and transform triangle normal.
    float3 normal = g_CubeAttribsCB.Normals[primitive.x].xyz * barycentrics.x +
        g_CubeAttribsCB.Normals[primitive.y].xyz * barycentrics.y +
        g_CubeAttribsCB.Normals[primitive.z].xyz * barycentrics.z;
    normal = normalize(mul((float3x3) ObjectToWorld3x4(), normal));

    // Air index of refraction
    const float AirIOR = 1.0;
    float3 resultColor = float3(0.0, 0.0, 0.0);

    // Enable dispersion - simulate rays with different wavelengths.
    // For optimization, disable dispersion after several reflections/refractions.
    if (g_ConstantsCB.GlassEnableDispersion && payload.Recursion == 0)
    {

```

```

float3 AccumColor = float3(0.0, 0.0, 0.0);
float3 AccumMask = float3(0.0, 0.0, 0.0);

RayDesc ray;
ray.TMin = SMALL_OFFSET;
ray.TMax = 100.0;

// Cast multiple rays with different wavelengths.
const int step = MAX_DISPERS_SAMPLES / g_ConstantsCB.DispersionSamp
for (int i = 0; i < MAX_DISPERS_SAMPLES; i += step)
{
    float3 norm = normal;
    float3 color;
    float rellOR = 1.0;

    // Calculate index of refraction for specified wavelength.
    float glassIOR = lerp(g_ConstantsCB.GlassIndexOfRefraction.x, g_Consta
    float3 colorMask = g_ConstantsCB.DispersionSamples[i].rgb; // RGB color
    float3 rayDir = WorldRayDirection();

    // Refraction at the interface between air and glass.
    if (HitKind() == HIT_KIND_TRIANGLE_FRONT_FACE)
    {
        rellOR = AirIOR / glassIOR;
        rayDir = refract(rayDir, norm, rellOR);
    }
    // Refraction at the interface between glass and air.
    else if (HitKind() == HIT_KIND_TRIANGLE_BACK_FACE)
    {
        rellOR = glassIOR / AirIOR;
        norm = -norm;
        rayDir = refract(rayDir, norm, rellOR);
    }

    float fresnel = Fresnel(rellOR, dot(WorldRayDirection(), -norm));
    float3 curColor = float3(0.0, 0.0, 0.0);

```

```

float3 reflColor;

// Reflection
{
    ray.Origin    = WorldRayOrigin() + WorldRayDirection() * RayTCurrent()
    ray.Direction = reflect(WorldRayDirection(), norm);

    PrimaryRayPayload reflPayload = CastPrimaryRay(ray, payload.RecursionDepth);
    reflColor = reflPayload.Color;

    if (HitKind() == HIT_KIND_TRIANGLE_BACK_FACE)
    {
        reflColor = LightAbsorption(reflColor, reflPayload.Depth);
    }
}

// Refraction
if (fresnel < 1.0)
{
    ray.Origin    = WorldRayOrigin() + WorldRayDirection() * RayTCurrent();
    ray.Direction = rayDir;

    PrimaryRayPayload nextPayload = CastPrimaryRay(ray, payload.RecursionDepth);
    curColor = nextPayload.Color;

    if (HitKind() == HIT_KIND_TRIANGLE_FRONT_FACE || payload.RecursionDepth == 0)
    {
        curColor = LightAbsorption(curColor, nextPayload.Depth);
    }
}

curColor    = BlendWithReflection(curColor, reflColor, fresnel);
AccumColor += curColor * colorMask;
AccumMask  += colorMask;
}

```

```

// Normalize accumulated color.
resultColor = AccumColor / AccumMask;
}
else
{
    RayDesc ray;
    ray.Direction = WorldRayDirection();
    ray.TMin      = SMALL_OFFSET;
    ray.TMax      = 100.0;

    float3 rayDir = WorldRayDirection();
    float rellOR = 1.0;

    // Refraction at the interface between air and glass.
    if (HitKind() == HIT_KIND_TRIANGLE_FRONT_FACE)
    {
        rellOR = AirIOR / g_ConstantsCB.GlassIndexOfRefraction.x;
        rayDir = refract(rayDir, normal, rellOR);
    }
    // Refraction at the interface between glass and air.
    else if (HitKind() == HIT_KIND_TRIANGLE_BACK_FACE)
    {
        rellOR = g_ConstantsCB.GlassIndexOfRefraction.x / AirIOR;
        normal = -normal;
        rayDir = refract(rayDir, normal, rellOR);
    }

    float fresnel = Fresnel(rellOR, dot(WorldRayDirection(), -normal));
    float3 reflColor;

    // Reflection
    {
        ray.Origin   = WorldRayOrigin() + WorldRayDirection() * RayTCurrent() + r
        ray.Direction = reflect(WorldRayDirection(), normal);

        PrimaryRayPayload reflPayload = CastPrimaryRay(ray, payload.Recursion

```

```

    reflColor = reflPayload.Color;

    if (HitKind() == HIT_KIND_TRIANGLE_BACK_FACE)
    {
        reflColor = LightAbsorption(reflColor, reflPayload.Depth);
    }
}

// Refraction
if (fresnel < 1.0)
{
    ray.Origin = WorldRayOrigin() + WorldRayDirection() * RayTCurrent();
    ray.Direction = rayDir;

    PrimaryRayPayload nextPayload = CastPrimaryRay(ray, payload.RecursionDepth + 1);
    resultColor = nextPayload.Color;

    if (HitKind() == HIT_KIND_TRIANGLE_FRONT_FACE || payload.RecursionDepth == 0)
    {
        resultColor = LightAbsorption(resultColor, nextPayload.Depth);
    }
}

resultColor = BlendWithReflection(resultColor, reflColor, fresnel);
}

payload.Color = resultColor;
payload.Depth = RayTCurrent();
}

```